

Studio Editor — Planning Research (v1.1)

Planning-only artifact. No application code. Sources: Canva Help, web research, open-source reference projects, product Q&A.

Canva animation model (design reference only)

Canva does **not** expose a public API for programmatic animation export. Designers apply presets per text element; timing is controlled via **element timing** on the video timeline.

Canva preset (design doc)	Observed behavior	Engine slug
Fade + Writing Style Element	Gradual opacity + letter/element reveal	fade_writing_element
Skate RTL + Writing Element	Text glides right-to-left while revealing	skate_writing_rtl
Burst + Writing	Pop/burst per letter on enter	burst_writing_enter
Speed slider (15% / 50%)	UI percentage, not ms	Calibrate durationMs vs reference MP4

Sources

- [Canva Help — Animate designs](#)
- [Canva Help — Edit element timing](#)

Frame-by-frame rule: At time t , each layer's visible glyphs = enter → static → exit. Before `enter.startMs` the layer is **fully invisible**. Writing animations reveal **per grapheme cluster** (Hindi-safe), not `string.length`.

Validated architecture

Browser: HTML video (1080p proxy) + DOM overlays + animation-engine → CSS Worker: animation-engine → node-canvas @ 60fps → RGB24 → FFmpeg → H.264 Both read: `definition_json` from Supabase

Rejected alternatives

Alternative	Why rejected
Supabase Edge + FFmpeg	Hosted Edge Runtime blocks shell / native FFmpeg
FFmpeg drawtext only	Devanagari shaping breaks without HarfBuzz/Pango; no per-glyph writing reveal
Remotion-only	Different runtime than DOM studio; commercial licensing; parity risk
Browser-only export	Weak Indic fonts, asset exposure, inconsistent MP4

Accepted stack

Vite/React 18 · Supabase (Auth/Postgres/Storage) · Fastify on Render · BullMQ+Redis · Docker worker (node-canvas+FFmpeg) · shared animation-engine · Razorpay INR · GA4

Technology compatibility matrix

Component	Works with	Constraint
Vite + React 18 SPA	Supabase JS, Razorpay.js	Lazy-load studio; fonts in public/
Supabase	Fastify, RLS	Worker uses service role for jobs
Fastify	BullMQ, Razorpay webhooks	Raw body for signature verify
node-canvas + FFmpeg	Same TS engine	Docker: ffmpeg, fontconfig, Noto Devanagari
pdf-lib PDF	Separate product	Static only in MVP

Reference projects (patterns, not dependencies)

Repo	Relevance
remotion-dev/remotion	Font load before headless frames

mifi/editly	JSON timeline + canvas + FFmpeg
pankod/canvas2video	Frames → FFmpeg over background video
Syrins/subtranslate	Supabase + queue + FFmpeg worker
Automattic/node-canvas	Pango / ctx.lang for Indic
harfbuzz/harfbuzzjs	Fallback shaping if browser ≠ server

Canvas + FFmpeg pipeline (industry pattern)

1. Spawn FFmpeg with `-f rawvideo -pix_fmt rgb24 -s WxH -r fps -i -`
2. Per frame: draw background frame + text on node-canvas
3. Write RGB24 buffer to `ffmpeg.stdin` with backpressure (drain)
4. `stdin.end()`; mux audio from source MP4 (`-c:a copy` where possible)

References: [DEV — Electron Canvas + FFmpeg](#), [canvas2video README](#).

Parity risks (preview vs export)

1. **Fonts** — same files in `@font-face` and `registerFont`
2. **Time** — integer ms; `frameIndex = floor(t_ms * fps / 1000)` in both paths
3. **Graphemes** — `Intl.Segmenter` or equivalent for Hindi/Latin
4. **Resolution** — positions in template canvas space; scale for 4K
5. **Alpha** — strip to RGB24 for H.264

Repository reference assets

File	Role
Engagement Sample 1 Timeline and Animation Details.docx	Layer positions, fonts, animation names
Sample Engagement Invitation 1.mp4	Golden QA reference

Shyara Digital - Studio Editor Developer
Brief.docx

Dev team handoff